FULL DSA ROADMAP:

Task1 : `Pick a language: Python`

Task2 : `Programming Fundamentals:`

`Pyhton was created by Guido van Rossum and released in 1991.`

`Python runs on an interpreter system, meaning that code can be executed as soon as it is written.`

`Python can be treated in a procedural way, an object-oriented way or a functional way.`

Python Syntax :

Pyhton Indentation : `it refers to the spaces in beginning of the code. it uses to indicate a block of code.`

`Python will give you an error if you skip the indentation.`

`Ex :`

```
if 5>2:
  print("yes")
```

```
yes
```

`The number of spaces is up to us common use is four, but it has to be at least one.`

`We have to use the same number of spaces in the same block of code, otherwise Python will give an error.`

Python Variables : `Variables are containers for storing data values.`

Python has no command for declaring a variable. A variable is created the moment you first assign a value to it.

Ex :

```
x = 'HIMANSHU'
y = 24
print(x)
print(y)
```

```
HIMANSHU
24
```

Variables do not need to be declared with any particular type, and can even change type after they have been set.

Casting : to specify the data type of a variable, this can be done with casting.

You can get the data type of a variable with the type() function.

Ex :

```
x = str(300) # there is a integer in variable but it will behave like a str as
print(type(x))
```

```
<class 'str'>
```

Variable names are case-sensitive. Means a is not equals to A.

Ex :

```
a = 4
A = 5
print(a, A)
```

```
4 5
```

```
A variable can have a short name (like x and y) or a more descriptive name (age, c
```

```
Rules for Python variables:
A variable name must start with a letter or the underscore character
A variable name cannot start with a number
A variable name can only contain alpha-numeric characters and underscores (A-z, 0-9
Variable names are case-sensitive (age, Age and AGE are three different variables)
A variable name cannot be any of the Python keywords.
```

Ex :

```
name = 'Himanshu'
na_me = 'Himanshu'
_name = 'Himanshu'
name1 = 'Himanshu'
print(name, na_me, name1, _name)
```

```
Himanshu Himanshu Himanshu Himanshu
```

Multiword Variable Name : Variable names with more than one word can be difficult to read. There are several techniques you can use to make them more readable.

Camel Case: Each word, except the first, starts with a capital letter.

Ex :

```
myName = 'Himanshu'
```

Pascal Case : Each word starts with a capital letter.

Ex :

```
MyName = 'Himanshu'
```

Snake Case : Each word is separated by an underscore character.

Ex :

```
my_name = 'Himanshu'
```

Assign Multiple Values : Python allows us to assign values to multiple variables in one line.

Make sure the number of variables matches the number of values, or will get an error.

Ex :

```
x, y = 'past', 'future'
print(x)
print(y)
```

```
past
future
```

We can assign the same value to multiple variables in one line.

Ex :

```
x = y = "Himanshu"
print(x)
print(y)
```

```
Himanshu
Himanshu
```

Unpack a Collection : If we have a collection of values in a list, tuple etc. Python allows you to extract the values into variables. This is called unpacking.

Ex :

```
fruits = ["apple", "banana", "cherry"]
x, y, z = fruits
print(x)
print(y)
print(z)
```

```
apple
banana
cherry
```

Output Variable : The print() function is often used to output variables.

Ex :

```
Name = "Himanshu" # A variable
print(Name) # Output Variable
```

```
Himanshu
```

In the print() function, we can output multiple variables, separated by a comma. We can also use the + operator to output multiple variables

+ operator don't add the space by itself as comma and for numbers it works as a mathematical operator. So don't use + operator to print number and string.

The best way to output multiple variables in the print() function is to separate them with commas, which even support different data types.

Ex :

```
name = "himanshu"
subject = "Maths"
print(name, subject)
print(name + subject)
print("Himanshu",24)
```

```
himanshu Maths
himanshuMaths
Himanshu 24
```

Global Variable : Variables that are created outside of a function are known as global variables. Global variables can be used by everyone, both inside of functions and outside.

Ex :

```
x = "Himanshu"
def name():
```

```
  print("My name is :", x)

name()
```

```
My name is : Himanshu
```

If you create a variable with the same name inside a function, this variable will be local, and can only be used inside the function. The global variable with the same name will remain as it was, global and with the original value.

Ex :

```
x = "awesome"

def myfunc():
  x = "fantastic"
  print("Python is " + x)

myfunc()

print("Python is " + x)
```

```
Python is fantastic
Python is awesome
```

Global Keyword : Normally, when you create a variable inside a function, that variable is local, and can only be used inside that function. To create a global variable inside a function, you can use the global keyword.

Ex :

```
def myfunc():
  global x
  x = "fantastic"

myfunc()

print("Python is " + x)
```

```
Python is fantastic
```

Also, use the global keyword if want to change a global variable inside a function.

Ex :

```
x = "awesome"

def myfunc():
  global x
  x = "fantastic"

myfunc()

print("Python is " + x)
```

```
Python is fantastic
```

Python Comments : Comments can be used to explain Python code, to make the code more readable, to prevent execution when testing code.

Comments starts with a #, and Python will ignore them.

Ex :

```
# it is a comment : by Himanshu
print("Thanks for the comment Himanshu")
```

```
Thanks for the comment Himanshu
```

To add a multiline comment you could insert a # for each line or we can use a multiline string.

Ex :

```
# Hi
# hello
print('Using hastags')
""" this is a
multiline
comment"""
print('Using multiline string')
```

```
Using hastags
Using multiline string
```

As long as the string is not assigned to a variable, Python will read the code, but then ignore it, and you have made a multiline comment.

Python Statements : A computer program is a list of instructions to be executed by a computer, these programming instructions are called statements.

Python programs contain many statements. The statements are executed one by one, in the same order as they are written.

Ex :

```
print('Hi')
print('Himanshu')
```

```
Hi
Himanshu
```

Python Output : We can use the print() function to display text or output values.

Text in Python must be inside quotes either " double quotes or ' single quotes. Otherwise python will error.

Ex :

```
print('My name is Himanshu')
```

```
My name is Himanshu
```

Print without a new line : mean by default, the print function ends with the new line to neglect this we can use end parameters.

Ex :

```

```

```
print('Hi,', end=' ')
print("My name is Himanshu")
```

```
Hi, My name is Himanshu
```

We can use print() function to print numbers but we don't require quotes anymore.

Ex :

```
print(3)
```

```
3
```

We can also do maths inside the print().

Ex :

```
print(2+3)
```

```
5
```

We can mix both text and numbers in same output by seprating them with a comma.

Ex :

```
print('Himanshu', 'age', 24)
```

```
Himanshu age 24
```

Python Datatypes : In programming, data type is an important concept. Variables can store data of different types, and different types can do different things

```
Python has the following data types built-in by default, in these categories:

Text Type:  str
Numeric Types:  int, float, complex
Sequence Types: list, tuple, range
```

```
Mapping Type:   dict
Set Types:  set, frozenset
Boolean Type:   bool
Binary Types:   bytes, bytearray, memoryview
None Type:  NoneType
```

`We can get the data type of any object by using the type() function.`

`Ex :`

```
x = 5050
print(type(x))
```

```
<class 'int'>
```

In Python, the data type is set when you assign a value to a variable.

```
a = "Hello World"
b = 20
c = 20.5
d = 1j
e = ["apple", "banana", "cherry"]
f = ("apple", "banana", "cherry")
g = range(6)
h = {"name" : "John", "age" : 36}
i = {"apple", "banana", "cherry"}
j = frozenset({"apple", "banana", "cherry"})
k = True
l = b"Hello"
m = bytearray(5)
n = memoryview(bytes(5))
o = None
print(type(a),type(b),type(c),type(d),type(e),type(f),type(g),type(h))
print(type(i),type(j),type(k),type(l),type(m),type(n),type(o))
```

```
<class 'str'> <class 'int'> <class 'float'> <class 'complex'> <class 'list'> <cl
<class 'set'> <class 'frozenset'> <class 'bool'> <class 'bytes'> <class 'bytearr
```

`Specifing Data Type : to specify the data type, we can use the constructor functions.`

Python Numbers : `There are three numeric types in Python: int, float, complex.`

Int, or integer, is a whole number, positive or negative, without decimals, of unlimited length.

Float, or "floating point number" is a number, positive or negative, containing one or more decimals. Float can also be scientific numbers with an "e" to indicate the power of 10.

Complex numbers are written with a "j" as the imaginary part:

Ex :

```
a = 1
b = 1.5
c = 1j
print(type(a))
print(type(b))
print(type(c))
```

```
<class 'int'>
<class 'float'>
<class 'complex'>
```

We can convert from one type to another with the int(), float(), and complex() methods.

Ex :

```
x = 1
y = 1.5
z = 1j
a = float(x)
b = int(y)
c = complex(x)
print(a)
print(b)
print(c)
print(type(a))
print(type(b))
print(type(c))
```

```
1.0
1
(1+0j)
```

```
<class 'float'>
<class 'int'>
<class 'complex'>
```

`Random Number : Python does not have a random() function to make a random number, but Python has a built-in module called random that can be used to make random numbers.`

`Ex :`

```
import random

num = random.random() # gives any random number.
print(num)

ran = random.randint(1, 10) # gives number between the given range.
print(ran)
```

```
0.03211484634592221
2
```

Python Casting : `it is used to specify a variable type.`

`Casting in python is done using constructor functions. Like : int(), float(), str().`

`Ex : int()`

```
x = int(1)
y = int(2.8)
z = int("3")
print(x,y,z)
```

```
1 2 3
```

`Ex : float()`

```
x = float(1)
y = float(2.8)
z = float("3")
w = float("4.2")

print(x,y,z,w)
```

```
1.0 2.8 3.0 4.2
```

Ex : str()

```
x = str("s1")
y = str(2)
z = str(3.0)
print(x,y,z)
```

```
s1 2 3.0
```

Python String : Strings in python are surrounded by either single quotation marks, or double quotation marks.

'hello' is the same as "hello". We can display a string literal with the print() function.

Ex :

```
print("Himanshu")
print('Himanshu')
```

```
Himanshu
Himanshu
```

We can use quotes inside a string, as long as they don't match the quotes surrounding the string.

Ex :

```
print("He is called 'Johnny'")
print('He is called "Johnny"')
```

```
He is called 'Johnny'
He is called "Johnny"
```

Assigning a string to a variable is done with the variable name followed by an equal sign and the string.

Ex :

```
x = 'Himanshu'
```

We can assign a multiline string to a variable by using three quotes single or double any.

Ex :

```
a = """Himanshu
is a
Cheeta"""
print(a)
```

```
Himanshu
is a
Cheeta
```

Strings are Arrays : Strings in Python are arrays of unicode characters.

Python does not have a character data type, a single character is simply a string with a length of 1.

Square brackets can be used to access elements of the string.

Ex :

```
greet = "Hi Himanshu"
print(greet[3])
```

```
H
```

Looping through String : Since strings are arrays, we can loop through the characters in a string, with a for loop.

Ex :

```
Name = "Himanshu"
for x in Name:
  print(x)
```

```
H
i
m
```

```
a
n
s
h
u
```

String Length : To get the length of a string, use the len() function.

Ex :

```
Name = "Himanshu"
print(len(Name))
```

```
8
```

Check String : To check if a certain phrase or character is present in a string, we can use the keyword in.

Ex :

```
Name = 'Himanshu'
print( 'a' in Name)
```

```
True
```

Check if not : To check if a certain phrase or char is NOT present in a string, we can use the keyword not in.

Ex :

```
Name = 'Himanshu'
print( 'a' not in Name)
```

```
False
```

Slicing Strings : We can return a range of characters by using the slice syntax.

Specify the start index and the end index, separated by a colon, to return a part of the string.

Ex :

```
Name = "Himanshu"
print(Name[2:5])
```

```
man
```

By leaving out the start index, the range will start at the first character.

Ex :

```
Name = "Himanshu"
print(Name[:5])
```

```
Himan
```

By leaving out the end index, the range will go to the end.

Ex :

```
Name = "Himanshu"
print(Name[2:])
```

```
manshu
```

Use negative indexes to start the slice from the end of the string.

Ex :

```
Name = "Himanshu"
print(Name[-5:-2])
```

```
ans
```

Modify Strings : Python has a set of built-in methods that we can use on strings.

1. Upper Case

Ex :

```
Name = "Himanshu"
print(Name.upper())
```

```
HIMANSHU
```

2. Lower Case

Ex :

```
Name = "Himanshu"
print(Name.lower())
```

```
himanshu
```

3. Remove WhiteSpaces

Ex :

```
Name = "  Himanshu Rajput "
print(Name.strip())
```

```
Himanshu Rajput
```

4. Replace Strings

Ex :

```
Name = "Himanshu"
print(Name.replace("H","S"))
```

```
Simanshu
```

5. Split Strings : The split() method returns a list where the text between the specified separator becomes the list items.

Ex :

```

```

```
Sen = "I am Himanshu"
print(Sen.split( ))
```

```
['I', 'am', 'Himanshu']
```

String Concatenation : To concatenate, or combine, two strings we can use the + operator.

Ex :

```
Name = "Himanshu"
SrName = "Rajput"
FullName = Name + SrName
print(FullName)
```

```
HimanshuRajput
```

To add a space between them, add a " "

Ex :

```
Name = "Himanshu"
SrName = "Rajput"
FullName = Name + " " + SrName
print(FullName)
```

```
Himanshu Rajput
```

Format String : As we learned in the Python Variables chapter, we cannot combine strings and numbers.

But we can combine strings and numbers by using f-strings or the format() method.

F-String was introduced in Python 3.6, and is now the preferred way of formatting strings.

To specify a string as an f-string, simply put an f in front of the string literal, and add curly brackets {} as placeholders for variables and other operations.

Ex :

```
txt = f"My name is Himanshu, I am {22}"
print(txt)
```

```
My name is Himanshu, I am 22
```

Placeholders and Modifiers :

A placeholder can contain variables, operations, functions, and modifiers to format the value.

Ex :

```
age = 22
txt = f"My name is Himanshu, I am {age}"
print(txt)
```

```
My name is Himanshu, I am 22
```

A placeholder can include a modifier to format the value.

A modifier is included by adding a colon : followed by a legal formatting type, like .2f which means fixed point number with 2 decimals.

Ex :

```
price = 59
txt = f"The price is {price:.2f} dollars"
print(txt)
```

```
The price is 59.00 dollars
```

A placeholder can contain Python code, like math operations.

```
txt = f"The price is {20 * 59} dollars"
print(txt)
```

```
The price is 1180 dollars
```

Python Escape Characters :

To insert characters that are illegal in a string, use an escape character.

An escape character is a backslash \ followed by the character you want to insert.

An example of an illegal character is a double quote inside a string that is surrounded by double quotes.

Ex :

```
txt = "We are the so-called \"Vikings\" from the north."
print(txt)
```

```
We are the so-called "Vikings" from the north.
```

Other escape characters used in Python:

```
\'    Single Quote
\\    Backslash
\n    New Line
\r    Carriage Return
\t    Tab
\b    Backspace
\f    Form Feed
\ooo    Octal value
\xhh    Hex value
```

Python String Methods : Python has a set of built-in methods that we can use on strings.

```
capitalize()    Converts the first character to upper case
casefold()  Converts string into lower case
center()    Returns a centered string
count() Returns the number of times a specified value occurs in a string
```

```
encode()    Returns an encoded version of the string
endswith()  Returns true if the string ends with the specified value
expandtabs()    Sets the tab size of the string
find()  Searches the string for a specified value and returns the position of where
format()    Formats specified values in a string
format_map()    Formats specified values in a string
index() Searches the string for a specified value and returns the position of where
isalnum()   Returns True if all characters in the string are alphanumeric
isalpha()   Returns True if all characters in the string are in the alphabet
isascii()   Returns True if all characters in the string are ascii characters
isdecimal() Returns True if all characters in the string are decimals
isdigit()   Returns True if all characters in the string are digits
isidentifier()  Returns True if the string is an identifier
islower()   Returns True if all characters in the string are lower case
isnumeric() Returns True if all characters in the string are numeric
isprintable()   Returns True if all characters in the string are printable
isspace()   Returns True if all characters in the string are whitespaces
istitle()   Returns True if the string follows the rules of a title
isupper()   Returns True if all characters in the string are upper case
join()  Joins the elements of an iterable to the end of the string
ljust() Returns a left justified version of the string
lower() Converts a string into lower case
lstrip()    Returns a left trim version of the string
maketrans() Returns a translation table to be used in translations
partition() Returns a tuple where the string is parted into three parts
replace()   Returns a string where a specified value is replaced with a specified v
rfind() Searches the string for a specified value and returns the last position of
rindex()    Searches the string for a specified value and returns the last position
rjust() Returns a right justified version of the string
rpartition()    Returns a tuple where the string is parted into three parts
rsplit()    Splits the string at the specified separator, and returns a list
rstrip()    Returns a right trim version of the string
split() Splits the string at the specified separator, and returns a list
splitlines()    Splits the string at line breaks and returns a list
startswith()    Returns true if the string starts with the specified value
strip() Returns a trimmed version of the string
swapcase()  Swaps cases, lower case becomes upper case and vice versa
title() Converts the first character of each word to upper case
translate() Returns a translated string
upper() Converts a string into upper case
zfill() Fills the string with a specified number of 0 values at the beginning
```

Python Boolean : Booleans represent one of two values: True or False.

We can evaluate any expression in Python, and get one of two answers, True or False. When we compare two values, the expression is evaluated and Python returns the Boolean answer.

Ex :

```
print(10 > 9)
print(10 == 9)
print(10 < 9)
```

```
True
False
False
```

When we run a condition in an if statement, Python returns True or False.

Ex :

```
a = 200
b = 33

if b > a:
  print("b is greater than a")
else:
  print("b is not greater than a")
```

```
b is not greater than a
```

The bool() function allows us to evaluate any value, and gives us True or False in return.

Ex :

```
print(bool("Hello"))
print(bool(15))
```

```
True
True
```

```
x = "Hello"
y = 15

print(bool(x))
print(bool(y))
```

```
True
True
```

Almost any value is evaluated to True if it has some sort of content.

Any string is True, except empty strings.

Any number is True, except 0.

Any list, tuple, set, and dictionary are True, except empty ones.

Ex :

```
bool(False)
bool(None)
bool(0)
bool("")
bool(())
bool([])
bool({})
```

```
False
```

We can create functions that returns a Boolean Value.

Ex :

```
def myFunction() :
  return True

print(myFunction())
```

```
True
```

Python also has many built-in functions that return a boolean value, like the isinstance() function, which can be used to determine if an object is of a certain data type.

Ex :

```
x = 200
print(isinstance(x, int))
```

```
True
```

Python Operators : `Operators are used to perform operations on variables and values.`

```
Python divides the operators in the following groups:

Arithmetic operators
Assignment operators
Comparison operators
Logical operators
Identity operators
Membership operators
Bitwise operators
```

Arithmetic Operators : `They are used with numeric values to perform common mathematical operations.`

```
+   Addition    x + y
-   Subtraction x - y
*   Multiplication  x * y
/   Division    x / y
%   Modulus x % y
**  Exponentiation  x ** y
//  Floor division  x // y
```

```
Python has two division operators:

/ - Division (returns a float)
// - Floor division (returns an integer)
```

`Ex :`

```

```

```
x = 12
y = 5

print(x / y)
print(x//y)
```

```
2.4
2
```

Assignment Operators : They are used to assign values to variables.

```
=       x = 5        x = 5
+=    x += 3       x = x + 3
-=    x -= 3       x = x - 3
*=    x *= 3       x = x * 3
/=    x /= 3       x = x / 3
%=    x %= 3       x = x % 3
//=   x //= 3      x = x // 3
**=   x **= 3      x = x ** 3
&=    x &= 3       x = x & 3
|=    x |= 3       x = x | 3
^=    x ^= 3       x = x ^ 3
>>=   x >>= 3      x = x >> 3
<<=   x <<= 3      x = x << 3
:=  print(x := 3)   x = 3print(x)
```

The Walrus Operator : Python 3.8 introduced the := operator, known as the "walrus operator". It assigns values to variables as part of a larger expression.

Ex :

```
numbers = [1, 2, 3, 4, 5]

if (count := len(numbers)) > 3:
    print(f"List has {count} elements")
```

```
List has 5 elements
```

Ternary Operators : They allows you to assign one value if a condition is true, and another if it is false.

Ex :

```
num = 6

x = "WEEKEND!" if num > 5 else "Workday"

print(x)
```

```
WEEKEND!
```

The ternary operator is not an actual operator, it is a conditional expression.

The ternary operator can be used instead of elif in longer if statements.

Ex :

```
num = 6

x = "Fri" if num == 5 else "Sat" if num == 6 else "Sun" if num == 7 else "weekc

print(x)
```

```
Sat
```

Comparison Operators : They are used to compare two values.

```
==  Equal
!=  Not equal
>   Greater than
<   Less than
>=  Greater than or equal to
<=  Less than or equal to
```

Comparison operators return True or False based on the comparison.

Ex :

```
x = 5
y = 3

print(x == y)
print(x != y)
print(x > y)
print(x < y)
print(x >= y)
print(x <= y)
```

```
False
True
True
False
True
False
```

`Chaining Comparison Operators : Python allows us to chain comparison operator.`

`Ex :`

```
x = 5

print(1 < x < 10)
print(1 < x and x < 10)
```

```
True
True
```

Logical Operations : `Logical operators are used to combine conditional statements.`

and	Returns True if both statements are true	x < 5 and x < 10
or	Returns True if one of the statements is true	x < 5 or x < 4
not	Reverse the result, returns False if the result is true	not(x < 5 and x < 1

`Ex :`

```
x = 5

print(x > 0 and x < 10)
```

```
True
```

Reverse the result with not.

Ex :

```
x = 5

print(not(x > 3 and x < 10))
```

```
False
```

Identity Operators : Identity operators are used to compare the objects, not if they are equal, but if they are actually the same object, with the same memory location.

is	Returns True if both variables are the same object	x is y
is not	Returns True if both variables are not the same object	x is not y

The is operator returns True if both variables point to the same object.

Ex :

```
x = ["apple", "banana"]
y = ["apple", "banana"]
z = x

print(x is z)
print(x is y)
print(x == y)
```

```
True
False
True
```

The is not operator returns True if both variables do not point to the same object.

Ex :

```
x = ["apple", "banana"]
y = ["apple", "banana"]

print(x is not y)
```

```
True
```

```
Difference between is and ==

is - Checks if both variables point to the same object in memory
== - Checks if the values of both variables are equal.
```

Ex :

```
x = [1, 2, 3]
y = [1, 2, 3]

print(x == y)
print(x is y)
```

```
True
False
```

Membership Operators : Membership operators are used to test if a sequence is presented in an object.

```
in      Returns True if a sequence with the specified value is present in the obje
not in  Returns True if a sequence with the specified value is not present in the
```

Ex :

```
fruits = ["apple", "banana", "cherry"]

print("banana" in fruits)
```

```
True
```

```
fruits = ["apple", "banana", "cherry"]

print("pineapple" not in fruits)
```

```
True
```

The membership operators also work with strings.

Ex :

```
text = "Hello World"

print("H" in text)
print("hello" in text)
print("z" not in text)
```

```
True
False
True
```

Bitwise Operators : Bitwise operators are used to compare (binary) numbers.

```
&   AND     Sets each bit to 1 if both bits are 1
|   OR      Sets each bit to 1 if one of two bits is 1
^   XOR     Sets each bit to 1 if only one of two bits is 1
~   NOT     Inverts all the bits
<<  Zero fill
    left shift  Shift left by pushing zeros in from the right and let the leftmost
>>  Signed right
        shift   Shift right by pushing copies of the leftmost bit in from the left,
                and let the rightmost bits fall off
```

Ex :

```
print(6 & 3)
```

```
2
```

```
print(6 | 3)
```

```
7
```

```
print(6 ^ 3)
```

```
5
```

Operator Precedence : Operator precedence describes the order in which operations are performed.

```
Operator     Description
()           Parentheses
**           Exponentiation
+x, -x, ~x   Unary plus, unary minus, and bitwise NOT
*, /, //, %  Multiplication, division, floor division, and modulus
+  -         Addition and subtraction
<<  >>       Bitwise left and right shifts
&            Bitwise AND
^            Bitwise XOR
|            Bitwise OR
(==, !=, >, >=, <, <=, is, is not, in, not in)  Comparisons, identity, and membersl
not          Logical NOT
and          AND
or           OR
```

If two operators have the same precedence, the expression is evaluated from left to right.

Ex :

```
print(5 + 4 - 7 + 3)
```

```
5
```

THANK YOU